

01_Data_Preparation: MF4 → Cleaned 10 ms Dataset

Validation_Aero_Test

August 4, 2026

1 Overview

This document describes the data-preparation pipeline that converts raw telemetry logs into a single analysis-ready CSV for downstream run extraction and drag calculation.

Goal Produce a fixed-sample-rate dataset (10 ms) with stable, documented column names.

Primary outputs

- Per-channel exports: `Outputs/01_Data_Preparation/Signals/*.csv`
- Canonical cleaned dataset: `Outputs/01_Data_Preparation/Aero_Validation_Signals_cleaned_10ms.csv`

Pipeline flow

```
Stint*.mf4
  |
  v
export_channels_10ms_to_signals.py  -->  Signals/<channel>.csv  (x N channels)
  |
  v
build_cleaned_signals_10ms.py      -->  Aero_Validation_Signals_cleaned_10ms.csv
  |
  v
load_data()  -->  sanitize_columns()  -->  add_derived_signals()
  |
  v
downstream (run extraction, drag calculation)
```

2 Inputs

- Raw logging data: `Data/<session>/Stint*.mf4`
- Channel list: `Data/channels_extraction.txt` (one channel name per line; # for comments)

3 Step 1: Export per-channel CSVs

Implemented in `Code/01_Data_Preparation/export_channels_10ms_to_signals.py`.

Concatenation strategy

Each `Stint*.mf4` is treated as a separate segment:

- Timestamps are shifted to start at 0 within the stint.
- Signals are interpolated onto a uniform grid with spacing Δt (default: 0.01 s).
- Stints are concatenated on a continuous time axis with an optional single- Δt gap between stints (enabled by default).

Per-channel output schema

For each requested channel c , the exporter writes:

- File: `Outputs/01_Data_Preparation/Signals/<channel>.csv`
- Columns: `time` (seconds) and `<channel_name>` (numeric; NaN outside signal support).

4 Step 2: Combine to the canonical cleaned dataset

Implemented in `Code/01_Data_Preparation/build_cleaned_signals_10ms.py`.

Join semantics

All per-channel CSVs are read and joined on the time column using an outer join. The resulting table is sorted by time; the index is reset and stored as column `t_s` (seconds).

Column sanitization

Raw MF4 channel names contain brackets, units, and occasional encoding artifacts (e.g., the degree symbol $^\circ$ may appear as a replacement character). The function `aero_validation.signals.sanitize_col` (called automatically by `load_data`) performs three operations:

1. Strips leading/trailing whitespace from all column headers.
2. Replaces mis-encoded characters with the correct symbol (e.g., replacement character $\rightarrow$ $^\circ$).
3. Maps known raw headers to canonical names via a static lookup table (defined in `_column_mapping`).
For example:

<code>ICU_Speedfl[km/h]</code>	$\rightarrow$	<code>ICU_Speedfl_kmh</code>
<code>SBG_IMU_ACCEL_X[m.s-2]</code>	$\rightarrow$	<code>IMU_ACCEL_X_ms2</code>
<code>ICU_SteeringAngle[$^\circ$]</code>	$\rightarrow$	<code>ICU_SteeringAngle_deg</code>

4. Drops unnamed trailing columns (e.g., empty columns from the logger).

All downstream code works exclusively with the sanitized canonical names.

Canonical column naming

The mapping in `_column_mapping` is the single source of truth for channel naming. If a new channel is added to the car, its raw header must be added to this mapping so that downstream scripts see a stable name. Names follow the convention `<Source>_<Signal>_<Unit>` (e.g., `GPS1_VEL_D_ms`).

5 Reproducibility

The following commands reproduce the full data-preparation pipeline from raw MF4 files (run from the repository root):

```
# Step 1: Export selected channels to per-channel CSVs at 10 ms
python Code/01_Data_Preparation/export_channels_10ms_to_signals.py \
    --stints-dir Data/<session> \
    --channels-file Data/channels_extraction.txt \
    --out-dir Outputs/01_Data_Preparation/Signals \
    --dt-ms 10

# Step 2: Combine per-channel exports into the canonical cleaned dataset
python Code/01_Data_Preparation/build_cleaned_signals_10ms.py \
    --channels-dir Outputs/01_Data_Preparation/Signals \
    --output Outputs/01_Data_Preparation/Aero_Validation_Signals_cleaned_10ms.csv
```

6 Derived signals

All derived signals are computed by the public wrapper `aero_validation.signals.add_derived_signals`, which delegates to the private sub-functions described below. Each sub-function modifies the DataFrame in place and can be tested independently.

6.1 Notation

The following time-synchronized columns are assumed to exist after sanitization:

- t [s]: timestamp `t_s`.
- $\mathbf{a}_{\text{IMU}} = (a_x, a_y, a_z)$ [m/s²]: IMU accelerations `IMU_ACCEL_X_ms2`, `IMU_ACCEL_Y_ms2`, `IMU_ACCEL_Z_ms2`.
- v_{gps} [m/s]: GPS speed magnitude `GPS1_VEL_MAG_ms`.
- v_{wheel} [m/s]: wheel-speed magnitude, converted from `WHEEL_SPEED_kmh` by $v_{\text{wheel}} = \text{km/h} \cdot \frac{1000}{3600}$.
- θ [deg]: steering angle `ICU_SteeringAngle_deg`.

6.2 Wheel speed

`_add_wheel_speed` computes `WHEEL_SPEED_kmh` as the row-wise mean of the four available wheel-speed channels (`ICU_Speedfl_kmh`, `ICU_Speedfr_kmh`, `ICU_Speedrl_kmh`, `ICU_Speedrr_kmh`). If some channels are missing, the mean is taken over the available subset.

6.3 GPS speed magnitude

`_add_gps_speed_magnitude` computes `GPS1_VEL_MAG_ms` from the three GPS velocity components when all are present:

$$v_{\text{gps}} = \sqrt{v_D^2 + v_E^2 + v_N^2}.$$

6.4 Zero-velocity mask

`_build_zero_velocity_mask` identifies samples where the vehicle is likely stationary. Two speed sources are combined with a strict threshold of 0.05 m/s:

$$\mathcal{Z} = \{ i \mid v_{\text{gps}}[i] < 0.05 \}$$

If wheel speed is also available, the mask is tightened to the conjunction:

$$\mathcal{Z} = \{ i \mid v_{\text{gps}}[i] < 0.05 \wedge v_{\text{wheel}}[i] < 0.05 \}.$$

6.5 IMU bias removal

Inside `_add_imu_longitudinal_accel`, per-axis stationary biases are estimated as medians over the zero-velocity mask $\mathcal{Z}$:

$$b_x = \text{median}\{ a_x[i] : i \in \mathcal{Z} \}, \quad b_y, b_z \text{ similarly.}$$

If $|\mathcal{Z}| < 20$ samples, the global median of each axis is used as a fallback. Bias-corrected accelerations are then

$$a'_x = a_x - b_x, \quad a'_y = a_y - b_y, \quad a'_z = a_z - b_z.$$

6.6 Smoothing and dv/dt

To obtain a stable numerical derivative of speed:

1. Compute the median sample period Δt_{med} from t and set a smoothing window of approximately 0.5 s ($\approx \frac{0.5}{\Delta t_{\text{med}}}$ samples, centered rolling mean).
2. Smooth the speed trace (prefer v_{gps} , fall back to v_{wheel}) to obtain $\tilde{v}(t)$.
3. Compute the numerical derivative:

$$\frac{dv}{dt}(t) \approx \text{gradient}(\tilde{v}(t), t).$$

Speed source preference. GPS speed is preferred because it measures ground-referenced velocity with low drift and is unaffected by tyre slip. Wheel speed is always available (no satellite dropouts) but can over- or under-report true speed during wheelspin or lockup. The fallback order ensures dv/dt is computed whenever at least one source is present.

6.7 Straight and moving segment masking

The axis-alignment weights (Section 6.8) are only fitted on samples where the vehicle is moving and approximately straight, to avoid lateral-acceleration contamination:

- Moving mask: $\tilde{v}(t) > 0.5 \text{ m/s}$.
- Straight mask: $|\theta(t)| \leq 5^\circ$ (if steering is available; otherwise skipped).

Samples must also have finite values in all required columns.

6.8 Longitudinal axis estimation (least-squares)

`_fit_accel_alignment_weights` estimates the longitudinal direction in the IMU frame by regressing bias-corrected IMU signals onto dv/dt :

$$\begin{aligned} A &= \begin{bmatrix} a'_x & a'_y & a'_z \end{bmatrix} \in \mathbb{R}^{N \times 3}, \\ y &= \frac{dv}{dt} \in \mathbb{R}^N, \\ \mathbf{w} &= \arg \min_{\mathbf{w} \in \mathbb{R}^3} \|A\mathbf{w} - y\|_2^2. \end{aligned}$$

If fewer than 50 valid samples remain after masking, the fit falls back to the identity assumption $\mathbf{w} = (1, 0, 0)$ (pure X-axis).

The longitudinal acceleration signal is then

$$a_{\text{long}}(t) = w_x a'_x(t) + w_y a'_y(t) + w_z a'_z(t),$$

stored as column `IMU_ACCEL_LONG_ms2`.

7 Notes

- **Resampling:** interpolation is performed independently per channel; NaN indicates missing data or out-of-range timestamps.
- **Bias removal:** the “stationary” mask uses a strict low-speed threshold (< 0.05 m/s) and falls back to global medians if too few stationary samples exist.
- **Longitudinal axis fit:** the least-squares weights are estimated only on straight + moving segments to avoid lateral acceleration contamination. If fewer than 50 valid samples remain after masking, the fit falls back to $\mathbf{w} = (1, 0, 0)$ (pure X-axis).
- **Missing channels:** if a requested channel is absent from the MF4 file, the pipeline produces a NaN-filled column rather than crashing. If multiple stints have different channel sets, the outer join fills missing segments with NaN.
- **GPS dropout:** the zero-velocity mask and dv/dt computation fall back to wheel speed when GPS is unavailable, ensuring the pipeline degrades gracefully rather than failing.
- **Column mapping maintenance:** when the logger configuration changes, update `_column_mapping` in `aero_validation/signals.py` to keep downstream code stable.

8 Example outputs

The figures below are diagnostic plots generated from the canonical cleaned dataset. They are produced by running the full data-preparation pipeline and then plotting selected time windows (these specific plots are generated ad-hoc and placed in `Outputs/01_Data_Preparation/plots/`).

The data shown is from the Steißlingen aero-validation test day (2025-10-22), comprising 4 stints and yielding approximately 46 coastdown runs after extraction.

9 Implementation references

- MF4 export: `Code/01_Data_Preparation/export_channels_10ms_to_signals.py`
- Combine & derived signals: `Code/01_Data_Preparation/build_cleaned_signals_10ms.py`

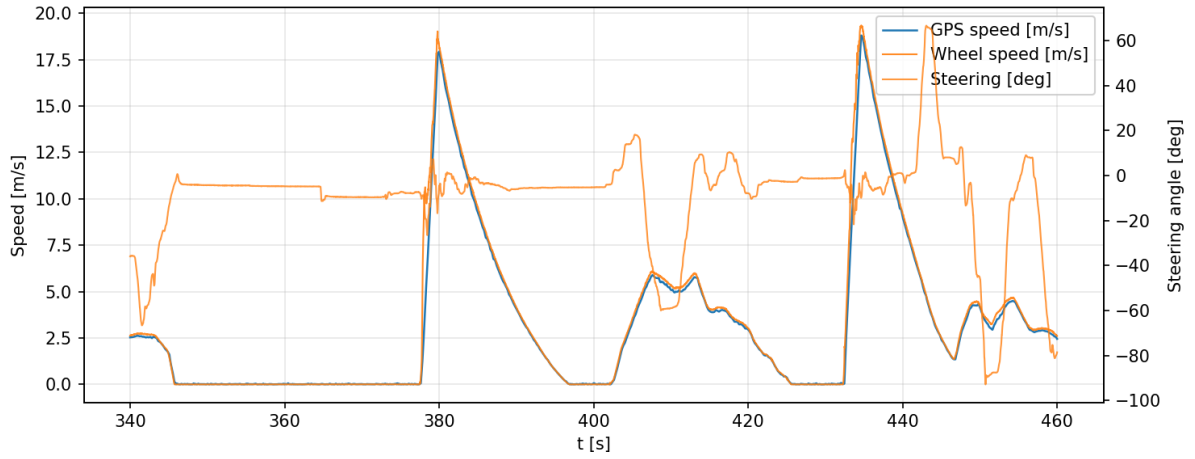


Figure 1: Example time window showing GPS speed, wheel speed (converted to m/s), and steering angle from the cleaned dataset.

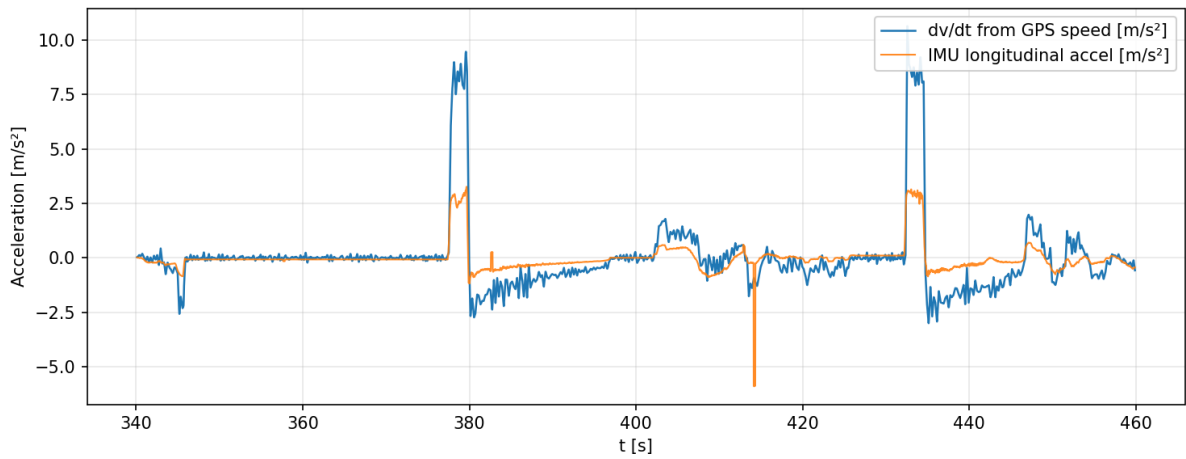


Figure 2: Comparison between dv/dt (computed from GPS speed) and the derived longitudinal IMU acceleration signal IMU_ACCEL_LONG.ms2.

- Column sanitization & mapping: `aero_validation/signals.py` – `sanitize_columns`, `_column_mapping`
- Derived signals (public wrapper): `aero_validation/signals.py` – `add_derived_signals`
- Sub-functions (individually testable):
 - `_add_wheel_speed`
 - `_add_gps_speed_magnitude`
 - `_build_zero_velocity_mask`
 - `_add_imu_longitudinal_accel`
 - `_fit_accel_alignment_weights`
- Data loading & path resolution: `aero_validation/io.py` – `load_data`, `data_path`, `repo_root`

The cleaned dataset produced by this pipeline is the input for the run extraction step (see 02.Run.Extraction.pdf) and ultimately for the drag coefficient estimation (see 03.Drag.Calculation.pdf).